

Appendix C: LLM-Driven Optimization

Anonymous

1 LLM-Based Parameter Optimization

In addition to traditional ASE tools, we investigated whether large language models (LLMs) can serve as effective alternatives for parameter optimization via direct model interaction. For this, we used the same seven benchmarks as in our **MAGPIE** and **ParamILS** experiments and employed **Qwen2.5-72B-Instruct**. This model achieves high performance on coding and reasoning benchmarks, often surpassing much larger models such as Llama-3.1-405B-Instruct despite being five times smaller and significantly more affordable [1]. Its open-weight release on Hugging Face and Alibaba Cloud ensures full reproducibility and community support, with competitive inference speed and cost-effective API pricing compared to proprietary alternatives [2].

1.1 Prompt Design

We crafted six distinct prompting strategies, each aimed at guiding the LLM to propose configurations that improve runtime. Example prompts are available at: **Results_LLM/Prompts**.

1. **Simple prompt:** Directly ask for an optimal parameter configuration.
2. **Chain-of-thought (CoT):** Encourage step-by-step reasoning before selecting parameters.
3. **Hint-based:** Provide general performance-improvement hints:
 - Replace inefficient algorithms with optimized alternatives.
 - Simplify logic while preserving correctness.
 - Eliminate unnecessary computations.
 - Enhance data structure usage for efficiency.
4. **Documentation prompt:** Supply detailed parameter documentation and their expected impact.
5. **CoT + documentation:** Combine reasoning prompts with parameter documentation.
6. **Hint + documentation:** Integrate optimization hints alongside documentation.

Each prompt was issued five times at two temperature settings (0.7 and 1.2), resulting in:

$$7 \times 6 \times 5 \times 2 = 420$$

total queries and corresponding parameter configurations.

1.2 Evaluation Process

All 420 configurations were first validated: configurations causing runtime errors or exceeding 40 s (benchmark baselines < 15 s) were discarded. For each valid configuration, we executed the benchmark 11 times and recorded the median runtime. The configuration with the lowest median per benchmark was deemed the “best” LLM-derived solution. These best configurations were then directly compared to those produced by **MAGPIE** to evaluate whether LLMs can match or exceed traditional ASE performance in parameter tuning.

1.3 Source Code Modification Attempts

We also explored LLM-driven source code modification for four benchmarks. However, prompt lengths commonly exceeded 15,000 tokens and output size limits (Qwen2.5-72B: 8,192 tokens; GPT-4: 16K+) prevented the model from returning full, compilable files. Even within token limits, generated edits were partial or non-compilable, requiring manual integration. As this violates full automation principles, we confined our LLM experiments to parameter optimization, which proved fully automatable and repeatable.

Through these experiments, we assess the viability of LLMs as standalone optimizers and compare prompting strategies, laying groundwork for future hybrid approaches that blend LLM inference with search-based methods.

2 Using Large Language Models for Automated Software Improvement

Additionally, we investigated whether LLMs can serve as effective alternatives to established automated software engineering tools by performing parameter optimization through direct LLM interaction.

2.1 Evaluation Process (Expanded)

For this experiment, we used the same seven benchmarks as in **MAGPIE** and **ParamILS** and performed parameter optimization using Qwen2.5-72B-Instruct. We designed six prompts (as above), each using a distinct prompting technique, and two temperatures (0.7 and 1.2) to generate diverse configurations. The prompts shared a common objective: generate parameter configurations that improve runtime performance.

All generated configurations were tested for validity. Configurations that produced run errors (e.g., wrong parameter values/combo) or exceeded 40s were discarded, as original execution times were < 15 s. For each valid configuration, we conducted 11 runtime measurements and retained the median as the representative performance metric. The optimal configuration for each benchmark was identified by the lowest median runtime. The best-performing LLM-derived configurations were then compared against those produced by **MAGPIE** to assess whether LLMs could effectively replace ASE tools for parameter tuning.

2.2 Cost Analysis

We also assessed token cost. On average, each query generated:

- **Output tokens per call:** 333 tokens, costing \$0.000133 per call.
- **Input tokens per call:** 1015 tokens, costing \$0.0002334 per call.
- Since each benchmark involved 60 calls, the per-benchmark cost was:
- **Output token cost per benchmark:** $60 \times 0.000133 = 0.008$ USD.
- **Input token cost per benchmark:** $60 \times 0.0002334 = 0.014$ USD.
- **Total cost per benchmark:** $0.008 + 0.014 = 0.022$ USD.

Across all seven benchmarks:

- **Output cost (7 benchmarks):** $7 \times 0.008 = 0.056$ USD.
- **Input cost (7 benchmarks):** $7 \times 0.014 = 0.098$ USD.

Thus, the total cost for all LLM queries:

$$\text{Total cost} = 0.056 + 0.098 = \mathbf{0.154 \text{ USD.}} \tag{1}$$

This cost is minimal compared to traditional computational methods; \$0.154 roughly equals a 200W laptop’s electricity cost for 6 hours at an average US kWh price of \$0.13 [3]. Hence, LLM-based optimization is a highly cost-effective alternative or complement to existing ASE tools.

2.3 Time Efficiency Comparison

Beyond cost, we considered wall-clock time. In our experiments, each LLM query took on average 12.33 s. Given 60 calls per benchmark:

$$60 \times 12.33 \approx \mathbf{12} \text{ minutes and } \mathbf{20} \text{ seconds.} \quad (2)$$

This is significantly lower than the 30-minute budget used in **MAGPIE** for parameter tuning, suggesting LLM-based optimization can operate within a fraction of the computational budget when rapid experimentation is needed.

2.4 Comparison of Prompt Effectiveness

We evaluated prompt effectiveness by analyzing the average runtime of configurations produced across all benchmarks and temperatures. Timeouts or runtime errors were penalized at 40 s. Table 1 reports average and median execution times per prompt. (Medians are less comparable due to differing baseline runtimes; averages better capture total optimization effect across benchmarks.)

Prompt	Average Runtime (ms)	Median Runtime (ms)
Simple prompt	17616.33	11465
With hints	16201.71	8057
With CoT	19666.73	12971
Simple with documentation	13263.70	6987
With hints and documentation	12493.15	6929
With CoT and documentation	11942.96	8123

Table 1: Average and median runtime for different prompts (lower is better).

Key observations: the simple prompt performed poorly (second-worst), indicating limited guidance is insufficient. Hints provided minor gains ($\sim 8\%$ faster on average). CoT alone performed worst (11.6% slower than simple on average), suggesting unguided reasoning can mislead. Documentation had the largest impact (29.5% faster on average vs. no documentation). The best prompt combined CoT with documentation, yielding the lowest average runtime. Hints + documentation slightly outperformed documentation alone (5.8%), indicating small added value. Overall, documentation is the primary driver.

2.5 Comparison of LLM-Based and MAGPIE Parameter Optimization

To assess effectiveness, we compare the best LLM-derived configuration (across prompts and temperatures) to the best **MAGPIE** fitness measure (independent of retries) over seven benchmarks. Table 2 shows average execution times and standard deviations.

The LLM-based approach is highly competitive: average runtime 5.53 s, comparable to **MAGPIE**’s time (5.54 s). Three **MAGPIE** measures beat the LLM: **weights** (5.06 s), **perf_cycles** (5.38 s), and **perf_L1_dcache_loads** (5.45 s). For two benchmarks (**MiniSAT_Hack**, **Weka**), the LLM found the best configuration of any method; for **MiniSAT_Hack** (with documentation), the LLM halved the best **MAGPIE** time. This likely reflects the large parameter space (25 tuned params): within a

Method	Average Runtime (s)	Standard Deviation
LLM	5.53	3.6821
MAGPIE (Time)	5.54	3.6649
MAGPIE (perf_time)	5.66	3.6575
MAGPIE (perf_instructions)	5.71	3.6878
MAGPIE (perf_cycles)	5.38	3.7477
MAGPIE (perf_cache_references)	5.68	3.6683
MAGPIE (perf_cache_misses)	5.64	3.6641
MAGPIE (perf_branches)	5.52	3.6981
MAGPIE (perf_branch_misses)	5.62	3.7189
MAGPIE (perf_cpu_clock)	7.30	3.5308
MAGPIE (perf_task_clock)	7.17	4.1462
MAGPIE (perf_faults)	6.75	3.7290
MAGPIE (weights)	5.06	2.3427
MAGPIE (energy)	5.78	3.2180
MAGPIE (perf_L1_dcache_loads)	5.45	3.7556

Table 2: Average runtime and standard deviation for LLM-based and MAGPIE parameter optimization across seven benchmarks (lower is better). For MAGPIE, one run per benchmark is performed; the average is taken across seven benchmarks.

30-minute budget, MAGPIE’s stepwise search changes fewer parameters, whereas the LLM can propose multi-parameter configurations. Higher standard deviations reflect aggregation across benchmarks with different base runtimes; averages better represent total runtime savings across tasks.

These findings suggest that while traditional search-based methods remain highly effective, LLMs provide a compelling alternative in scenarios where:

- The parameter space is large and exhaustive stepwise optimization is impractical.
- Detailed documentation is available to guide the LLM’s decisions.
- A quick, near-optimal solution is preferred over extensive incremental search.

Thus, incorporating LLMs into automated software improvement workflows—especially for parameter tuning in highly configurable systems—can provide substantial time and efficiency benefits. Future work could explore hybrid approaches integrating LLMs with traditional search-based techniques to leverage strengths of both.

LLMs (Qwen2.5-72B-Instruct) can generate competitive parameter configurations—achieving 5.53s average across seven benchmarks, on par with MAGPIE’s best traditional measures (e.g., **Time** at 5.54s). Prompt design is critical: pure “simple” or CoT prompts perform worst (>19s average), while including explicit parameter documentation dramatically improves results. The best prompt combines CoT with documentation, reducing average runtime across all variants to 11.94s. Overall, LLMs excel when guided by detailed documentation and can outperform or match search-based methods in large, highly configurable spaces.

Note: This appendix consolidates all LLM-related details (design, evaluation, cost/time analysis, prompt effectiveness, and comparison with MAGPIE) for completeness.

References

- [1] Qwen Team. Qwen2.5 Technical Report. 2025. <https://arxiv.org/abs/2501.00000> (placeholder; replace with the official citation you use).
- [2] Artificial Analysis. Qwen2.5-72B-Instruct Benchmark Summary and Pricing Snapshot. 2025. <https://artificialanalysis.ai/models/qwen2.5-72b-instruct> (use the exact source you cited).
- [3] U.S. Energy Information Administration (EIA). Average Retail Price of Electricity. 2025. <https://www.eia.gov/electricity/monthly/> (insert specific page/date you referenced).